Introduction to Operations Research, Deterministic Models

Lecture 10: Solving Integer Programming Problems

Department of Industrial Engineering, Tsinghua University

Instructor: Junyi Liu

Nov 19nd, 2024

Recap: branch and bound algorithm

1. To start, derive a lower bound L with the corresponding feasible solution that is referred to as the incumbent solution.
2. Put the original problem on the candidate list.
3. Select a subproblem S_i from the candidate list and solve the LP relaxation to obtain the upper bound $b(S_i)$.
   - If the LP relaxation is infeasible $\Longrightarrow$ node can be pruned.
   - If $b(S_i) \leq L \Longrightarrow$ node can be pruned.
   - If $b(S_i) > L$ and the solution is feasible for the MILP $\Longrightarrow$ set $L \leftarrow b(S_i)$ and update the incumbent solution.
   - Otherwise ($b(S_i) > L$ and the solution is infeasible), branch and add the new subproblems to the candidate list.
4. If the candidate list is nonempty, go to Step 3. Otherwise, the algorithm is completed.

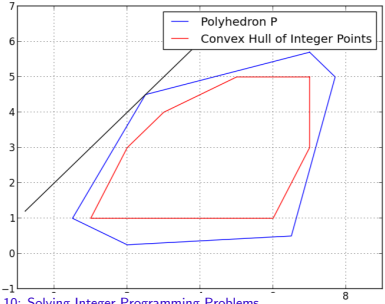
The incumbent solution is the best solution on hand.

In this lecture, you'll learn

- Branch-and-bound method
  - Importance of formulation
  - Solving MIP with near optimality
- Cutting plane method
- Branch and cut method

Importance of Formulation

- A typical MILP can have many alternative equivalent formulations.
- Each formulation corresponds to a different polyhedron enclosing the integer points that are feasible for the problem.
- Formulations may lead to different upper bounds of LP relaxation, so choosing the right one is critical in order to provide "good" upper bounds.
- The more closely the polyhedron approximates the convex hull of the integer solutions, the better the bound will be.



Example: Facility location problem

- We are given n potential facility locations and m customers.
- There is a fixed cost c_j of opening facility j.
- There is a cost d_{ij} associated with serving customer i from facility j.
- We have two sets of binary variables.
  - y_j is 1 if facility j is opened, 0 otherwise.
  - x_{ij} is 1 if customer i is served by facility j, 0 otherwise.

Example: Facility location problem

$$
\begin{array}{lll}
\text{minimize} & \displaystyle\sum_{j=1}^{n} c_j y_j + \sum_{i=1}^{m}\sum_{j=1}^{n} d_{ij} x_{ij} & \\
\text{s.t.} & \displaystyle\sum_{j=1}^{n} x_{ij} = 1, & \forall i \\
& \displaystyle\sum_{i=1}^{m} x_{ij} \leq m y_i, & \forall j \\
& x_{ij} \in \{0,1\}, y_j \in \{0,1\}, & \forall i,j
\end{array}
$$

$$
\begin{array}{lll}
\text{minimize} & \displaystyle\sum_{j=1}^{n} c_j y_j + \sum_{i=1}^{m}\sum_{j=1}^{n} d_{ij} x_{ij} & \\
\text{s.t.} & \displaystyle\sum_{j=1}^{n} x_{ij} = 1, & \forall i \\
& x_{ij} \leq y_i, & \forall i,j \\
& x_{ij} \in \{0,1\}, y_j \in \{0,1\}, & \forall i,j
\end{array}
$$

- Notice that the set of integer solutions contained in each of the polyhedra is the same.
- However, the second polyhedra is strictly included in the first one.
- The second polyhedra will yield stronger bounds and be more suitable for BB algorithm.
- Notice that the second formulation includes more constraints, but will likely solve more quickly by providing better bounds.

Solving near optimal solutions

With much less computational effort, the branch-and-bound technique can be used to find a nearly optimal solution, whose objective value is "close enough" to the optimal value Z^*.

A feasible solution x is said to be near optimal if its objective value Z satisfies:

$$Z \geq Z^* - \varepsilon, \quad \text{or} \quad Z \geq (1-\gamma)Z^*$$

with a very small $\varepsilon, \gamma \in (0,1)$, e.g., 0.05.

In most problems, the optimal value Z^* is difficult to be attained. If we can obtain a good upper bound of Z^*, denoted by UB, and

$$Z \geq UB - \varepsilon, \quad \text{or} \quad Z \geq (1-\gamma)UB$$

then it implies that x is near optimal.

Solving near optimal solutions

In branch-and-bound method, let L be the objective value of the incumbent solution. Consider the subproblem S_i.

- With the goal of attaining the optimal solution, a node is pruned if $b(S_i) \leq L$.
- If the goal is to attain the near optimal solution, a node is pruned if

$$b(S_i) - \varepsilon \leq L, \quad \text{or} \quad (1-\gamma)b(S_i) \leq L.$$

Why such a node can be pruned?

- The incumbent is close enough to the subproblem's optimal solution so that the subproblem does not need to be considered further.
- When there are no remaining subproblems, the current incumbent will be the desired nearly optimal solution.
- This will expedite the algorithmic procedure for extremely large problems in practice, and produce a nearly optimal solution which way may be sufficient for practical purposes.

Cutting plane method

Cutting plane method: background

- The cutting-plane algorithm solves integer programs by modifying linear-programming solutions until the integer solution is obtained. It was the first algorithm developed for integer programming that could be proved to converge in a finite number of steps.

- It does not partition the feasible region into subdivisions, as in branch-and-bound approaches. Instead, it works with a single linear program successively, which is refined by adding new constraints. The new constraints successively reduce the feasible region until an integer optimal solution is found.

- In practice, the algorithm generally is considered to be very inefficient, and the branch-and-bound procedures almost always outperform the cutting-plane algorithm. Nevertheless, it has provided insights into integer programming that have led to other, more efficient, algorithms (e.g., branch-and-cut methods).

Cutting plane method: example

Consider an integer program

$$\begin{array}{ll} \text{maximize} & 5x_1 + 8x_2 \\ \text{subject to} & x_1 + x_2 \leq 6 \\ & 5x_1 + 9x_2 \leq 45 \\ & x_1 \geq 0, x_2 \geq 0 \\ & x_1, x_2 \text{ integer} \end{array}$$

Its LP relaxation problem:

$$\begin{array}{ll} \text{maximize} & 5x_1 + 8x_2 \\ \text{subject to} & x_1 + x_2 + s_1 = 6 \\ & 5x_1 + 9x_2 + s_2 = 45 \\ & x_1 \geq 0, x_2 \geq 0 \\ & s_1 \geq 0, s_2 \geq 0 \end{array}$$

Cutting plane method: example

Solving the LP relaxation problem produces:

$$
\begin{array}{lllll}
z & & & +\frac{5}{4}s_1 + \frac{3}{4}s_2 = 41\frac{1}{4} \\
 & x_1 & & +\frac{9}{4}s_1 - \frac{1}{4}s_2 = \frac{9}{4} \\
 & & x_2 & -\frac{5}{4}s_1 + \frac{1}{4}s_2 = \frac{15}{4} \\
 & x_1 \geq 0, & x_2 \geq 0, & s_1 \geq 0, \quad s_2 \geq 0
\end{array}
$$

Rewrite these equations in a way that involves integer coefficients to the lefthand side of the equalities and fractions to the righthand side with nonnegative constant terms and nonpositive coefficients for the variables.

Cutting plane method: example

Rewritten equations:

$$
\begin{aligned}
z \qquad\qquad\qquad\quad & +s_1 + 0s_2 - 41 = \frac{1}{4} - \frac{1}{4}s_1 - \frac{3}{4}s_2 \\
x_1 \qquad\qquad & +2s_1 - s_2 - 2 = \frac{1}{4} - \frac{1}{4}s_1 - \frac{3}{4}s_2 \\
x_2 \quad & -2s_1 + 0s_2 - 3 = \frac{3}{4} - \frac{3}{4}s_1 - \frac{1}{4}s_2 \\
x_1 \geq 0, \quad x_2 \geq 0, \quad & s_1 \geq 0, \quad s_2 \geq 0
\end{aligned}
$$

Observations:

• In any integer solution, the lefthand side of each equation in the last tableau must be integer.

• Since s_1 and s_2 are nonnegative and appear to the right with negative coefficients, each righthand side necessarily must be less than or equal to the fractional constant term.

They imply that both sides of every equation must be an integer less than or equal to zero:

$$\frac{1}{4} - \frac{1}{4}s_1 - \frac{3}{4}s_2 \leq 0, \quad \frac{1}{4} - \frac{1}{4}s_1 - \frac{3}{4}s_2 \leq 0, \quad \frac{3}{4} - \frac{3}{4}s_1 - \frac{1}{4}s_2 \leq 0$$

Cutting plane method: example

$$\frac{1}{4} - \frac{1}{4}s_1 - \frac{3}{4}s_2 \leq 0, \quad \frac{3}{4} - \frac{3}{4}s_1 - \frac{1}{4}s_2 \leq 0$$

The above inequalities are called cuts with the following two properties:
- the current optimal solution to the LP relaxation does not satisfy the cut;
- any integer feasible point to the original problem must satisfy the cut constraints, so that the cuts do not exclude any integer solutions to the problem.

The value of a cut: to cut away the optimal linear-programming solution from the feasible region without excluding any feasible integer points.

An optimal solution to the integer-programming problem will be obtained, after adding only a finite number of cuts by iteratively solving LP relaxation problems via the dual simplex method.

Branch-and-cut method

Branch-and-cut method

Cutting planes methods were very first algorithms (1958) developed for integer programming, but this approach proved to be unsatisfactory in practice.

For MIP problems with many thousand variables, Branch-and-bound methods usually encounter an explosion in computation time.

Branch-and-cut (mid-1980s) is an LP-based branch-and-bound scheme in which the LP relaxations are augmented by cutting planes. By judiciously combining cutting planes and branch-and-bound techniques, it provides a powerful algorithmic approach for solving large-scale BIP problems (e.g., thousand variables).

The approach mainly uses techniques:

- **Automatic problem preprocessing**
- **Perform bounding with the generation of cutting planes**

Automatic problem preprocessing

1. **Fixing variables:**

Identify variables that can be fixed at one of their possible values (either 0 or 1) because the other value cannot possibly be part of a solution that is both feasible and optimal.

e.g. $3x_1 + x_2 \leq 2 \quad \Longrightarrow \quad x_1 = 0$

General procedure: within a $\leq$ constraint, identify the variable with the largest positive coefficient, and if the sum of that coefficient and any negative coefficients exceeds the right-hand side, then that variable should be fixed at 0.

e.g. $5x_1 + x_2 - 2x_3 \leq 2 \quad \Longrightarrow \quad x_1 = 0$

2. **Eliminating redundant constraints:** Identify and eliminate redundant constraints (constraints that automatically are satisfied by solutions that satisfy all the other constraints).

How? For a $\leq$ constraint, if even the most challenging binary solution (variables equal to 1 for those associated with nonnegative coefficients and other variables equal to 0) satisfies that constraint, then it is redundant and can be eliminated from further consideration.

Bounding with cutting planes

In the branch-and-cut method, the bounding loop is essentially a cutting plane method for solving the subproblem S_i in the candidate list, but with some kind of early termination criteria.

• The bounding loop for subproblem S_i consists of several steps applied iteratively until termination criteria is met.

– solve the current LP relaxation.
– decide whether node can be fathomed (by infeasibility or bound).
– generate valid inequalities violated by the solution to the LP relaxation.
– perform primal heuristics.
– apply node pre/post-processing.
– manage/improve LP relaxation (add/remove cuts, change bounds)

• After termination, branching is performed to continue the algorithm.

Bounding with cutting planes

In practice, the number of inequalities generated can be HUGE.

- We must be careful to keep the size of the LP relaxations small or we will sacrifice efficiency.

- This is done in two ways:
– Limiting the number of cuts that are added each iteration.
– Systematically deleting cuts that have become ineffective.

Dynamic Programming

Dynamic Programming

Dynamic programming is a useful mathematical technique for **making a sequence of interrelated decisions**.

• In contrast to linear programming, there does not exist a standard mathematical formulation of "the" dynamic programming problem. Rather, dynamic programming is a general type of approach to problem solving, and the particular equations must be developed to fit each situation.

• Dynamic programming starts with a small portion of the original problem and finds the optimal solution for this smaller problem. It then gradually enlarges the problem, finding the current optimal solution from the preceding one, until the original problem is solved in its entirety.

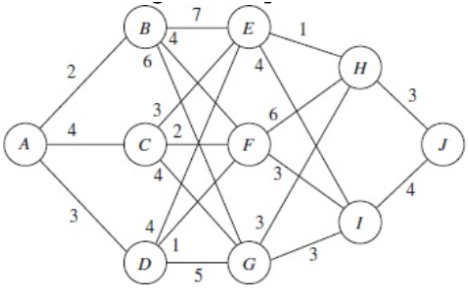
• These abilities to recognize when and how a problem can be solved by dynamic programming procedures can best be developed by an exposure to a wide variety of dynamic programming applications and a study of the characteristics that are common to all these situations.

Example: The Stagecoach Problem

The Stagecoach Problem concerns a mythical fortune seeker in Missouri who decided to go west to join the gold rush in California during the mid-19th century.

His starting point is Missouri and destination point is California, and he had considerable choice as to which states (or territories that subsequently became states) to travel through en route.

The direction of travel is always from left to right in the diagram. Thus, four stages (stagecoach runs) were required to travel from his point of embarkation in state A (Missouri) to his destination in state J (California).

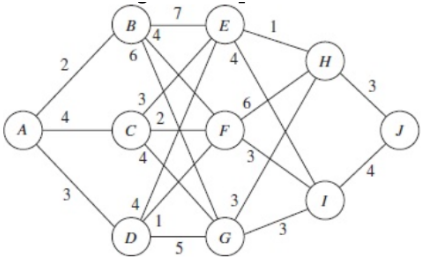


The cost of each run (e.g., traveling cost and life insurance costs) is given in the left figure.

Question: which route minimizes the total cost of the policy?

Example: the Stagecoach Problem

There are four stages of decisions, and the corresponding costs are summarized below.



Stage 1:

State	B	C	D
A	2	4	3

Stage 2:

State	E	F	G
B	7	4	6
C	3	2	4
D	4	1	5

Stage 3:

State	H	I
E	1	4
F	6	3
G	3	3

Stage 4:

State	J
H	3
I	4

Example: the Stagecoach Problem

- The trivial problem: if the fortune seeker is at stage 3 and has only one more stage (stagecoach run) to go. The obvious optimal solution for this smaller problem is to go from his current state (whatever it is) to his ultimate destination (state J).
- The enlarged problem: the problem is enlarged by increasing an additional number of stages left to go to complete the journey.
- The recursion: when the fortune seeker is at stage 2, from each possible state at stage 2 the optimal solution for where to go next can be found relatively easily from the results obtained at the preceding iteration.

Example: the Stagecoach Problem

- Let the decision variables x_n for $n = 1, 2, 3, 4$ be the immediate destination on stage n. Thus, the route selected is $A \to x_1 \to x_2 \to x_3 \to x_4$, where $x_4 = J$.

- Given that the fortune seeker is in state s at stage $n-1$, let $f_n^*(s)$ be the total cost of the best overall policy **for the remaining stages**.

- Let $W(s)$ denote the candidates of the states to go from the current state s.

$$f_n^*(s) = \min_{x_n \in W(s)} c_{s,x_n} + f_{n+1}^*(x_n)$$

- Let $x_n^*(s)$ denote the optimal solution of x_n in the above minimization problem, which represents the best state to go from the current state s to the ultimate destination.

- $f_1^*(A)$ can be found by successively finding $f_4^*(s)$, $f_3^*(s)$, $f_2^*(s)$, for each of the possible states s in each stage.

Solution procedure

- Stage 4:

$f_4^*(H) = c_{H,J} = 3$ and $f_4^*(I) = c_{I,J} = 4$.

- Stage 3:

$f_3^*(E) = \min\{c_{E,H} + f_4^*(H), c_{E,I} + f_4^*(I)\} = \min\{1+3, 4+4\} = 4$ and $x_3^*(E) = H$

$f_3^*(F) = \min\{c_{F,H} + f_4^*(H), c_{F,I} + f_4^*(I)\} = \min\{6+3, 3+4\} = 7$ and $x_3^*(F) = I$

$f_3^*(G) = \min\{c_{G,H} + f_4^*(H), c_{G,I} + f_4^*(I)\} = \min\{3+3, 3+4\} = 6$ and $x_3^*(G) = H$

- Stage 2:

$f_2^*(B) = \min\{c_{B,E} + f_3^*(E), c_{B,F} + f_3^*(F), c_{B,G} + f_3^*(G)\} = \min\{7+4, 4+7, 6+6\} = 11$ and $x_2^*(B) = E/F$

$f_3^*(C) = \min\{c_{C,E} + f_3^*(E), c_{C,F} + f_3^*(F), c_{C,G} + f_3^*(G)\} = \min\{3+4, 2+7, 3+6\} = 7$ and $x_2^*(C) = E$

$f_3^*(D) = \min\{c_{D,E} + f_3^*(E), c_{D,F} + f_3^*(F), c_{D,G} + f_3^*(G)\} = \min\{4+4, 1+7, 5+6\} = 8$ and $x_2^*(D) = E/F$

Characteristics of DP

- The problem can be divided into **stages**, with a policy decision required at each stage. Each stage has a number of **states** associated with the beginning of that stage. The number of states may be either finite or infinite.

The stagecoach problem was literally divided into its four stages (stagecoaches) that correspond to the four legs of the journey. The states associated with each stage in the stagecoach problem were the territories in which the fortune seeker could be located, and the policy decision at each stage was which territory to select for the next stagecoach ride.

Characteristics of DP

• The effect of the policy decision at each stage is to transform the current state to a state associated with the beginning of the next stage.

The fortune seeker's decision as to his next destination led him from his current state to the next state on his journey.

•The solution procedure is designed to find an optimal policy for the overall problem, i.e., a prescription of the optimal policy decision at each stage for each of the possible states.

The solution procedure constructs the optimal decision $(x_n^*(s))$ for each possible state s at each stage n. Besides the optimal solutions for the overall problem, the results show the fortune seeker how he should proceed if he gets detoured to a state that is not on an optimal route.

Characteristics of DP

• Given the current state, an optimal policy for the remaining stages is independent of the policy decisions adopted in previous stages. Therefore, knowledge of the current state of the system conveys all the information about its previous behavior necessary for determining the optimal policy henceforth. This is the **principle of optimality** for dynamic programming.

Given the state in which the fortune seeker is currently located, the optimal route from this point onward is independent of how he got there.

Characteristics of DP

• The solution procedure begins by finding the optimal policy for the last stage. A recursive relationship identifies the optimal policy for stage n, given that the optimal policy for stage $n+1$ is available.

The recursive relationship $f_n^*(s) = \min_{x_n \in W(s)} c_{s,x_n} + f_{n+1}^*(x_n)$. Finding the optimal policy decision $x_n^*(s)$ requires finding the minimizing value of x_n with the minimal values $f_{n+1}^*(x_n)$ in previous iterations. At stage n, the optimal route starting from state s is $x_n^*(s)$ followed by the optimal policy when you start in state $x_n^*(s)$ at stage $n+1$.

• When we use this recursive relationship, the solution procedure starts at the end and moves backward stage by stage—each time finding the optimal policy for that stage—until it finds the optimal policy starting at the initial stage.

Reading materials

Section 5.3, 5.4, 5.8, *Operations Research Mathematical Programming,* by Wayne Winston.

Section 12.5-12.8, *Introduction to Operations Research,* by Hillier and Lieberman